FPGA BASED AUTONOMOUS SHIP FOR REAL TIME NAVIGATION AND OBJECT DETECTION

A project report submitted in partial fulfillment of the requirements for the award of

Bachelor of Technology

In

ELECTRONICS & COMMUNICATION ENGINEERING

By

SANKARASETTI PRABHAS (22BQ1A04C8)

SIMHADRI JALANDHAR (22BQ1A04D6)

SUVARNAGANTI LEELA NIKHIL (22BQ1A04D8)

UNDER THE GUIDANCE OF

Prof. M. Y. BHANU MURTHY

DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING

VASIREDDY VENKATADRI INSTITUTE OF TECHNOLOGY

(AUTONOMOUS)

(Approved by AICTE and permanently affiliated to JNTUK)

Accredited by NBA and NAAC with 'A' Grade

NAMBUR (V), PEDAKAKANI (M), GUNTUR-522 508

APRIL 2026

DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING

VASIREDDY VENKATADRI INSTITUTE OF TECHNOLOGY: NAMBUR

(AUTONOMOUS)

JAWAHARLAL NEHRU TECHNOLOGICAL UNIVERSITY KAKINADA

CERTIFICATE

This is to certify that the project titled "**FPGA BASED AUTONOMOUS SHIP FOR REAL TIME NAVIGATION AND OBJECT DETECTION**" is a bonafide record of work done by **Mr. SANKARASETTI PRABHAS (22BQ1A04C8), Mr. SIMHADRI JALANDHAR (22BQ1A04D6)** and **Mr. SUVARNAGANTI LEELA NIKHIL (22BQ1A04D8)** under the guidance of **Prof. M. Y. BHANU MURTHY** in partial fulfillment of the requirement of the degree for Bachelor of Technology in Electronics and Communication Engineering, JNTUK during the academic year 2025–26.

Prof. M.Y. BHANU MURTHY
Project Guide

Dr. Sk. Enaul Haq
Head of the Department

DECLARATION

We, **SANKARASETTI PRABHAS (22BQ1A04C8), SIMHADRI JALANDHAR (22BQ1A04D6)**, And **SUVARNAGANTI LEELA NIKHIL (22BQ1A04D8),** hereby declare that the Project Report entitled "**FPGA BASED AUTONOMOUS SHIP FOR REAL TIME NAVIGATION AND OBJECT DETECTION** " done by us under the guidance of **Prof. M. Y. Bhanu murthy**, **Professor**, **Department of ECE** is submitted in partial fulfillment of the requirements for the award of degree of **BACHELOR OF TECHNOLOGY** in **ELECTRONICS AND COMMUNICATION ENGINEERING**.

DATE :

PLACE: VVIT, Nambur

SIGNATURE OF THE CANDIDATES

SANKARASETTI PRABHAS

SIMHADRI JALANDHAR

SUVARNAGANTI LEELA NIKHIL

SSiT SOLUTIONS PVT. LTD.

SENSE SEMICONDUCTOR & IT SOLUTIONS PVT. LTD.

Mangalagiri, Andhra Pradesh

An ISO 9001:2015 Certified Organization

CERTIFICATE OF COMPLETION

This is to certify that

MR./MS. **SANKARASETTI PRABHAS**

with Student ID **22BQ1A04C8** a student of **Vasireddy Venkatadri Institute of Technology - Nambur** has successfully completed Internship in **FPGA based VLSI Design** with project title **FPGA BASED AUTONOMOUS SHIP FOR REAL TIME NAVIGATION AND OBJECT DETECTION** at SSIT Solutions Pvt. Ltd. for the duration from **26th August 2024** to **26th March 2026**. During this period, the candidate demonstrated strong commitment, professionalism, and a willingness to learn. They were punctual, sincere, and carried out the assigned tasks with dedication and efficiency. We appreciate their efforts and wish them great success in all their future endeavors.

Certificate No.

SSIT-2026- 0 5 2 6

Issue Date: 28/03/2026

Ms. Lavanya Redy
Human Resourse Manager
SSIT Solutions Pvt. Ltd.

Mr. Surya Trinadh Chadalsetty
Chief Operating Officer
SSIT Solutions Pvt. Ltd.

ACKNOWLEDGEMENT

We express our sincere thanks wherever it is due

We express our sincere thanks to the Chairman, Vasireddy Venkatadri Institute of Technology, Sri Vasireddy Vidya Sagar for providing us well equipped infrastructure and environment.

We thank Dr. Y. Mallikarjuna Reddy, Principal, Vasireddy Venkatadri Institute of Technology, Nambur, for providing us the resources for carrying out the project.

We express our sincere thanks to Dr. K. Giribabu, Dean of Academics for providing support and stimulating environment for developing the project.

Our sincere thanks to Dr. SK. Enaul Haq, Head of the Department, Department of ECE, for his co-operation and guidance which helped us to make our project successful and complete in all aspects.

We also express our sincere thanks and are grateful to our guide Prof. M. Y. BHANUMURTHY, Professor, Department of ECE, for motivating us to make our project successful and fully complete. We are grateful for his precious guidance and suggestions.

We also place our floral gratitude to all other teaching staff and lab technicians for their constant support and advice throughout the project.

NAME OF THE CANDIDATES

SANKARASETTI PRABHAS (22BQ1A04C8)
SIMHADRI JALANDHAR (22BQ1A04D6)
SUVARNIGANTI LEELA NIKHIL(22BQ1A04D8)

TABLE OF CONTENTS

LIST OF TABLES

LIST OF FIGURES

ABSTRACT

Maritime transport carries billions of tons of cargo every year and plays a critical role in the global economy. It is one of the most efficient and cost-effective modes of transportation for international trade. However, despite technological advancements, a large number of maritime accidents still occur, mainly due to human errors such as poor decision-making, fatigue, and lack of real-time awareness. These accidents can lead to heavy financial losses, environmental damage, and safety risks.

Traditional ship navigation systems depend largely on manual control and high-power computing systems. These systems are not only expensive but also consume a significant amount of energy, making them less suitable for continuous and large-scale deployment. Moreover, they often lack real-time responsiveness and efficient integration of multiple sensors.

With the rapid development of embedded systems, artificial intelligence, and machine learning, autonomous navigation systems are emerging as a promising solution to improve maritime safety and efficiency. Autonomous ships are capable of sensing their surroundings, detecting obstacles, and making intelligent decisions without human intervention. This reduces the chances of human error and improves overall navigation performance.

In this project, an FPGA-based autonomous ship prototype is developed to perform real-time object detection and navigation. The system utilizes the FPGA and Raspberry Pi 4B FPGA board as the core processing unit. A webcam is used to capture real-time images of the environment, while ultrasonic sensors provide distance measurements to nearby objects. The GPS module continuously tracks the ship's location, enabling accurate path planning and navigation.

The FPGA architecture supports parallel processing, allowing multiple operations such as image processing, object detection, and route planning to be executed simultaneously. This results in faster computation, reduced latency, and lower power consumption compared to conventional CPU or GPU-based systems. Due to these advantages, FPGA-based systems are highly suitable for real-time and energy-efficient maritime applications.

CHAPTER – 1
INTRODUCTION

Maritime transport is the backbone of the global economy, accounting for the transportation of billions of tons of cargo annually. It is the most cost-effective and efficient method for international trade, connecting continents and sustaining global supply chains. Despite its significance, the maritime industry faces persistent challenges, primarily concerning safety and operational efficiency. Statistical data indicates that a vast majority of maritime accidents—ranging from ship collisions to groundings—are the direct result of human error during navigation. These errors often stem from fatigue, limited visibility, or delayed decision-making in complex environments.

Traditional ship navigation systems have historically relied on manual control and heavy human supervision. While these systems incorporate radar and electronic charts, the final decision-making process remains human-centric. Furthermore, conventional autonomous attempts often require high-power computing systems (CPUs and GPUs) which are bulky, consume excessive energy, and may struggle with the millisecond-level latency required for real-time obstacle avoidance in unpredictable water conditions.

With the rapid advancement of embedded systems and Artificial Intelligence (AI), autonomous navigation has emerged as a transformative solution to improve maritime safety. An autonomous ship is designed to perceive its surroundings, analyze environmental data, and execute navigation commands without constant human intervention. By integrating advanced sensors and intelligent algorithms, these systems can detect obstacles—such as other vessels, debris, or landmasses—and calculate optimal, safe routes in real-time.

The primary objective of this project is to design and implement an **FPGA-based Autonomous Ship Prototype**. Unlike standard PC-based controllers, this system leverages the hardware acceleration capabilities of Field Programmable Gate Arrays (FPGAs).

- **Hardware Platform:** The project utilizes the **FPGA and Raspberry Pi 4B FPGA board**, which provides a robust SoC (System-on-Chip) environment.
- **Sensor Integration:** To achieve comprehensive environmental awareness, the system integrates:
    - **Camera Sensors:** For visual object recognition and horizon detection.
    - **Ultrasonic Sensors:** For short-range proximity sensing and collision avoidance.

- **GPS Module:** For precise localization and waypoint navigation.

The core strength of this project lies in the FPGA architecture. Conventional CPU-based systems process instructions sequentially, which can create bottlenecks during intensive tasks like image processing. In contrast, the FPGA allows for **Parallel Processing**, enabling the system to handle data from multiple sensors simultaneously.

- **High Performance:** Faster computation for real-time object detection.
- **Low Power Consumption:** FPGAs offer higher performance-per-watt compared to GPUs, making them ideal for battery-operated or solar-powered maritime prototypes.
- **Reliability:** Hardware-level execution reduces the risk of software crashes, ensuring consistent navigation performance in critical environments.

CHAPTER – 2

LITERATURE REVIEW

2.1 Review of Existing Autonomous Navigation Systems

Many existing autonomous navigation systems rely on high-performance CPUs or GPUs to execute complex computer vision algorithms. While these systems are capable of handling advanced computations, they consume a large amount of power and generate significant heat. This makes them less suitable for embedded maritime environments where energy efficiency and reliability are critical factors. Additionally, high computational requirements increase the overall cost and limit their deployment in small-scale or resource-constrained systems.

Traditional navigation systems also suffer from a lack of proper integration between multiple sensors such as cameras, ultrasonic sensors, and GPS modules. In many cases, these sensors operate independently without effective coordination. As a result, the system may fail to obtain a complete understanding of the surrounding environment. Without proper sensor fusion, it becomes difficult to achieve accurate object detection, distance measurement, and reliable navigation, especially in complex or dynamic maritime conditions.

In recent years, research has focused on incorporating machine learning techniques into autonomous navigation systems. Algorithms like YOLO (You Only Look Once) have proven to be highly effective for real-time object detection, enabling the identification of ships, buoys, and obstacles with good accuracy. Similarly, path planning algorithms such as A* are widely used to determine optimal routes based on parameters like distance, obstacles, and environmental conditions. These approaches significantly improve the intelligence and efficiency of autonomous systems.

However, implementing these advanced algorithms on FPGA platforms offers additional advantages. FPGAs support parallel processing, allowing multiple tasks such as image preprocessing, object detection, and path planning to be executed simultaneously. This leads to faster response times and reduced latency, which are essential for real-time navigation. Moreover, FPGA-based systems consume significantly less power compared to traditional CPU and GPU-based systems, making them ideal for energy-efficient applications.

2.1.1 *EXISTING ARCHITECTURE*

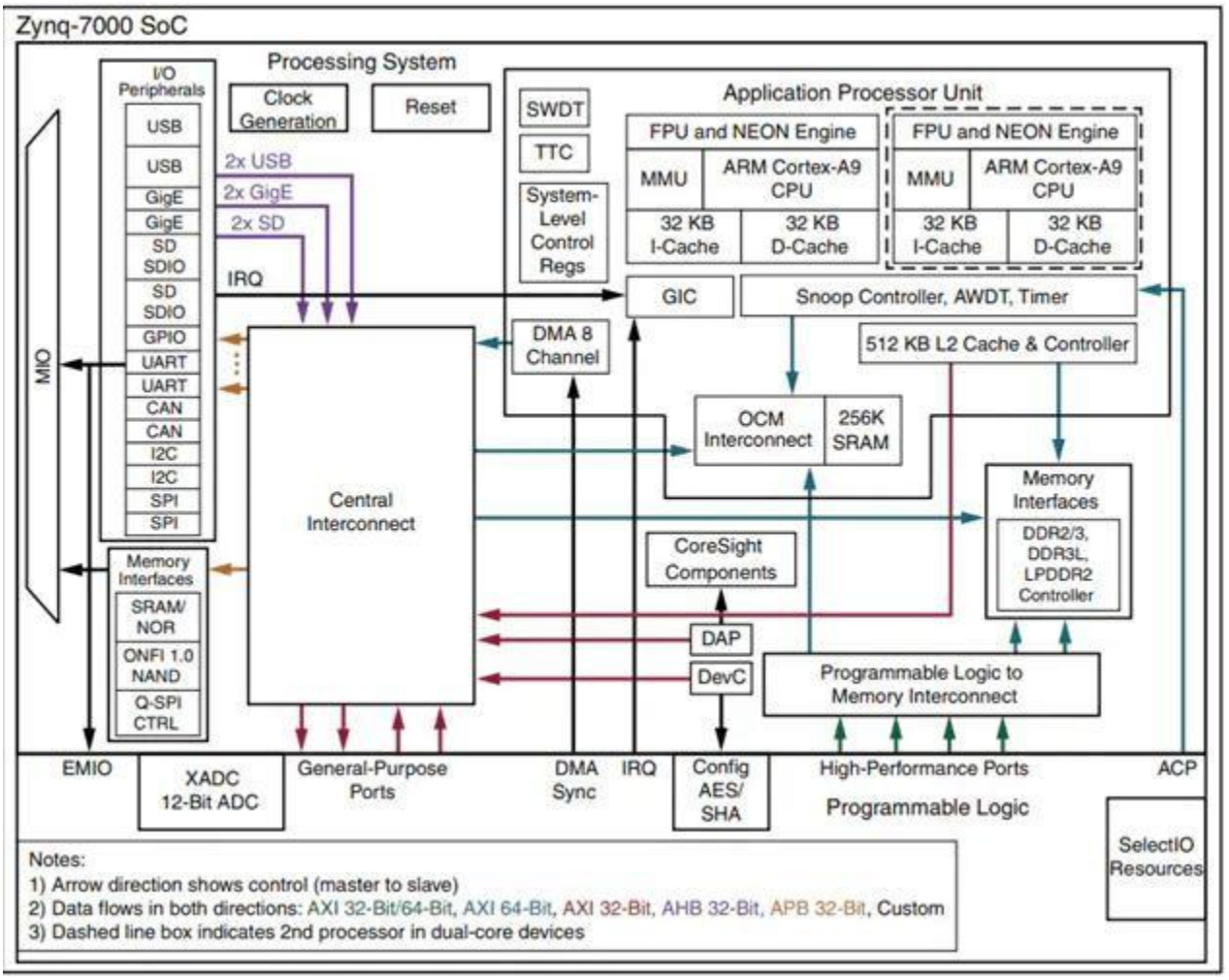


Figure 2.1 The Hardware Architecture Of EDGE FPGA

The figure represents the internal architecture of an FPGA-based System-on-Chip (SoC), typically similar to the Xilinx Zynq platform. This architecture combines a Processing System (PS) with Programmable Logic (PL), enabling both software and hardware co-design.

The system integrates processors, memory, peripherals, and configurable logic into a single chip, allowing efficient real-time processing and flexibility.

- **The processor executes instructions and controls system operations.**
- **Data is stored and accessed through on-chip or external memory.**
- **Peripherals communicate via standard interfaces (UART, SPI, etc.).**
- **Programmable Logic accelerates computation-intensive tasks.**
- **Interconnect ensures smooth communication between all components.**

CHAPTER 3
PROPOSED SYSTEM

3.1 PROPOSED SYSTEM

The suggested single cycle processor has many number of LUTs which produce complex design. In present system we use fewer LUT's than in other existing systems. Single cycle microarchitecture has two interactive components. Data path and control unit, Data path works on words of data control unit provides control signals to help data path to do right things at right time. We first design a microarchitecture that executes instructions in a single cycle. We begin constructing the datapath by connecting the state elements with combinational logic that can execute the various instructions. Control signals determine which specific instruction is performed by the datapath at any given time. The control unit contains combinational logic that generates the appropriate control signals based on the current instruction. Finally, we analyze the performance of the single-cycle processor. The single-cycle processor's control unit computes the control signals based on op, funct3, and funct7. For the RV32I instruction set, only bit 5 of funct7 is used, so we just need to consider op (Instr6:0), funct3 (Instr14:12), and funct75 (Instr30). To simplify decoding, the instruction encoding is highly regular. Basic instructions are 32 bits long and are naturally aligned. The designers where very careful to ensure consistency between formats. For example, the opcode and operands are in fixed locations, simplifying the decode process. The source and destination registers fields are of 5 bits for decoding one of the 32 registers. There are basically six types of instructions as is shown in the following table.

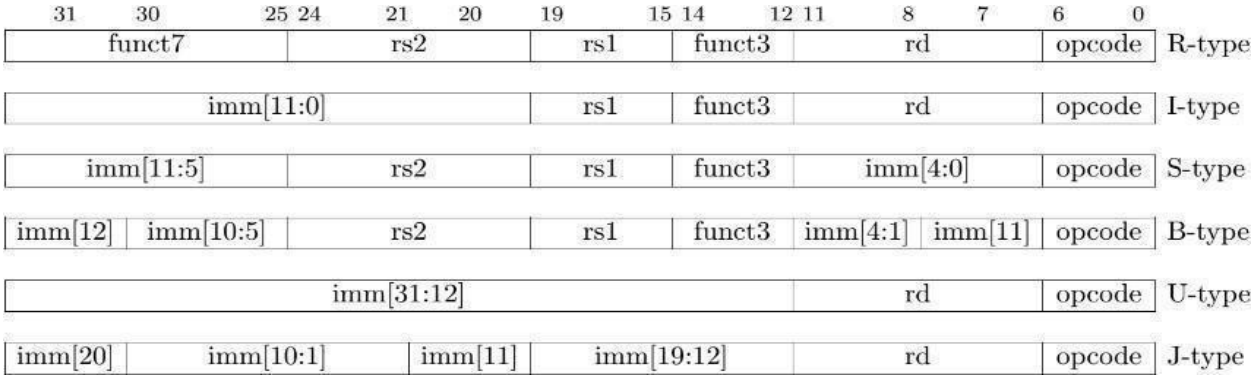


Table 3.1 Instruction format

3.2 ARCHITECTURE OF PROPOSED SYSTEM

1. EDGE FPGA Board

The EDGE FPGA board is responsible for hardware acceleration and low-level processing.

- It performs compute-intensive tasks such as image processing and object detection (e.g., YOLOv3-tiny).
- Directly interfaces with sensors like camera modules and ultrasonic sensors.
- Processes raw data in parallel, enabling high-speed execution.
- Handles low-level signal processing and preprocessing of sensor data.
- Reduces latency by offloading heavy computation from the processor.

2. Raspberry Pi

The Raspberry Pi acts as the main processing and control unit.

- Runs an operating system (Linux-based).
- Manages high-level communication protocols such as MQTT.
- Performs navigation and decision-making tasks.
- Interfaces with cloud platforms for data transmission and monitoring.
- Communicates with the FPGA board to receive processed data.

3. Hardware-Software Co-Design

This system follows a **co-design approach**:

- **EDGE FPGA Board** → Handles time-critical and parallel tasks
- **Raspberry Pi** → Handles control logic and communication

Data is exchanged between them using communication interfaces like:

- UART
- SPI
- or Ethernet

4. Working Principle

1. Sensors (camera, ultrasonic) capture real-time data.
2. EDGE FPGA processes the data (e.g., object detection).
3. Processed results are sent to the Raspberry Pi.
4. Raspberry Pi performs navigation and decision-making.
5. Data is transmitted to cloud or control systems via MQTT.

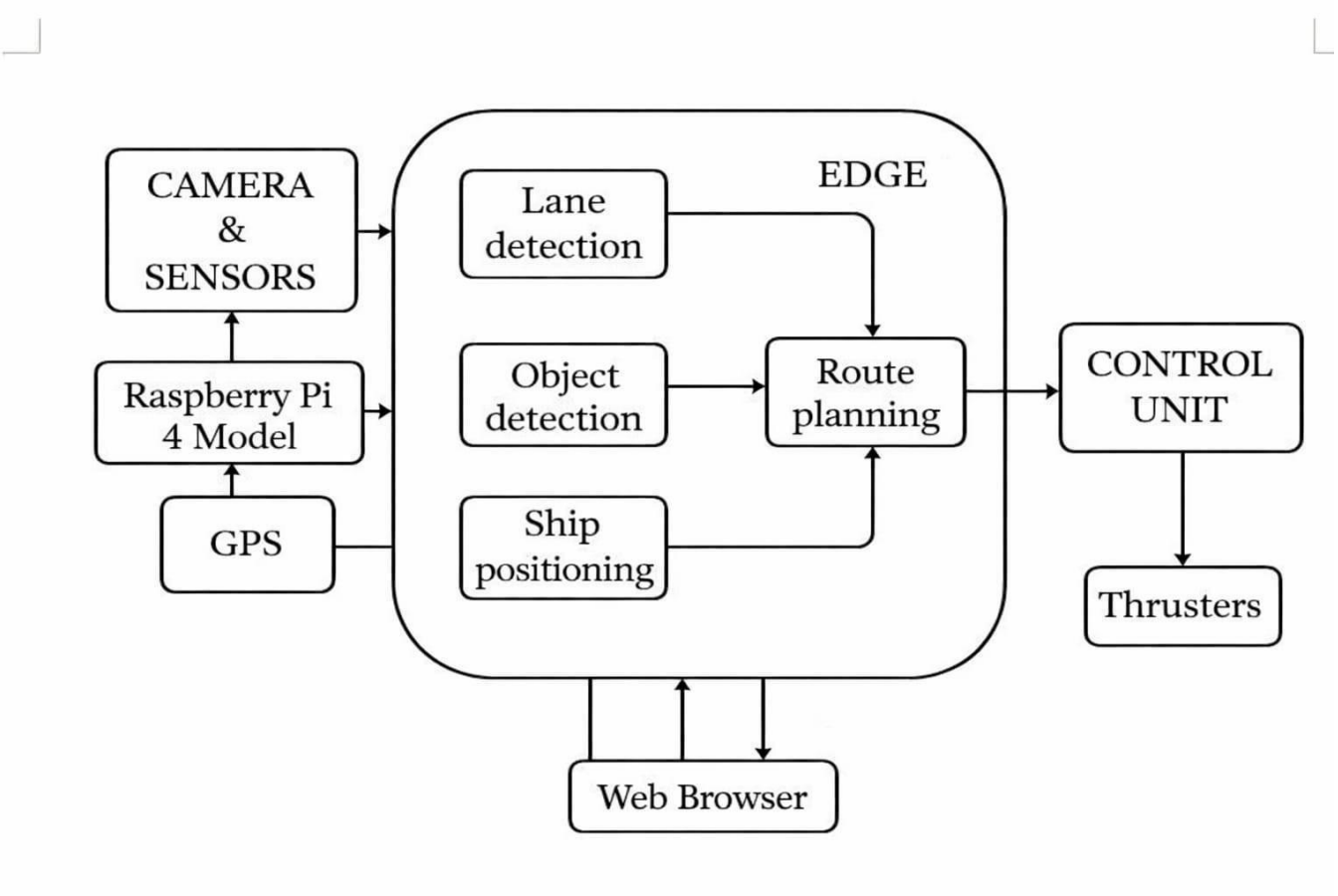


FIGURE 3.1: Flow chat of Execution flow

GPS Module

- Provides real-time location coordinates (latitude & longitude).
- Helps in navigation and tracking the ship's position.

Object Detection

- Detects obstacles like ships, rocks, or floating objects.
- Uses camera input and algorithms (e.g., YOLO).
- Ensures collision avoidance.

Planning Unit

- Makes decisions based on:
- GPS data
- Detected obstacles
- Calculates the optimal path for navigation.
- Avoids obstacles while reaching the destination

Control Unit

- Converts planning decisions into actions.
- Controls:
  - Motors
  - Direction (steering)
- Ensures smooth and safe movement of the ship.

3.3 CONTROL UNIT:

The "vision" of the autonomous ship is facilitated by a high-definition camera module.

- **YOLOv3-tiny Algorithm:** To achieve real-time object detection on an embedded platform, we implement the YOLOv3-tiny (You Only Look Once) neural network. Unlike conventional algorithms that scan images multiple times, YOLO treats detection as a single regression problem, significantly reducing latency.
- **Detection Capabilities:** The system is trained to recognize maritime-specific objects such as other ships, navigational buoys, and shorelines. The FPGA processes these frames in parallel, allowing the ship to "see" and "classify" obstacles simultaneously.

Opcode	Jump	Branch	ToRegister	MemWrite	StoreSel	ALUSrc	WriteReg
R-type (0110011)	0	000	000	0	0	1	1
I-type (0010011)	0	000	000	0	0	0	1
I-store (0100011)	0	000	000	1	0-SW 1-SB	0	0
I-load (0000011)	0	000	001-LB 010-LW	0	0	0	1
Branch (1100011)	0	001-BEQ 010-BNQ 100-BLT 101-BGT	000	0	0	1	0
JALR (1100111)	1	101	011	0	0	1	0
JAL (1101111)	0	110	101	0	0	1	0

Table 3.2: control signal generated based on instruction formats

Instruction	Funct 3	Funct 7	ALUOP	Operation
R-Type	000	0000000	100	ADD
	000	0100000	101	SUB
	001		110	SLL
	010		011	SLT
	100		010	XOR
	101		111	SRL
	110		010	OR
	111		000	AND
I – Type Arith	000		100	ADDI
	111		000	ANDI
	100		010	XORI
	110		100	ORI
I – Type Load	000		100	LB
	010		100	LW
I – Type Store	000		100	SB
	010		100	SW
Branch	000		101	BEQ
	001		101	BNQ
	100		101	BLT
	101		101	BGT

Table 3.3: Based on instructions the operations are followed In ALU

3.4 SOFTWARE AND PROGRAMMING LANGUAGES

While the camera provides long-range vision, **Ultrasonic Sensors** are integrated for short-range proximity sensing and redundancy.

- **Distance Measurement:** These sensors emit high-frequency sound pulses and measure the time-of-flight (ToF) for the echo to return. This data is critical in low-visibility conditions (like fog or heavy rain) where visual data might be unreliable.
- **Safety Buffer:** The system maintains a pre-defined "Safety Zone." If an obstacle is detected within this perimeter by the ultrasonic sensors, the FPGA immediately triggers a collision avoidance routine, overriding the current path to prevent impact.

FIGURE 3.2 XILINX VIVADO LOGO

3.4.1 VIVADO

Vivado was introduced in April 2012, and is an integrated design environment (IDE) with system-to-IC level tools built on a shared scalable data model and a common debug environment. Vivado includes electronic system level (ESL) design tools for synthesizing and verifying C-based algorithmic IP; standards-based packaging of both algorithmic and RTL IP for reuse; standards-based IP stitching 26 and systems integration of all types of system building blocks; and the verification of blocks and systems. A free version WebPACK Edition of Vivado provides designers with a limited version of the design environment.

Vivado includes electronic system level (ESL) design tools for synthesizing and verifying C-based algorithmic IP; standards-based packaging of both algorithmic and RTL IP for reuse; standards-based IP stitching and systems integration of all types of system building blocks; and the verification of blocks and systems.

3.4.2 VHDL Programming

VHDL is a short form of VHDL Hardware Description Language where VHSIC stands for Very High-Speed Integrated Circuits. It's a hardware description language means it describes the behavior of a digital circuit, and also it can be used to derive or implement a digital circuit/system hardware. It can be used for digital circuit synthesis as well as simulation. It is used to build digital system/circuit using Programmable Logic Device like CPLD (Complex Programmable Logic Device) or FPGA (Field Programmable Gate Array). VHDL program (code) is used to implement digital circuit inside CPLD / FPGA, or it can be used to fabricate ASIC (Application Specific Integrated Circuit).

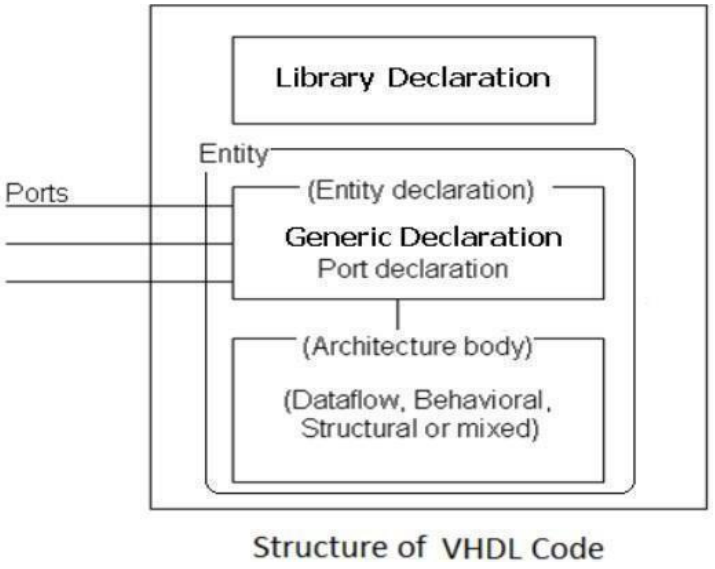


FIGURE 3.3 STRUCTURE OF VHDL CODE

It is very useful in developing high end, sophisticated microprocessor or micro-controller like ASIP (Application Specific Instruction Processor) or PSoC (Programmable System on Chip).

The VHSIC Hardware Description Language (VHDL) is a hardware description language (HDL) that can model the behavior and structure of digital systems at multiple levels of abstraction, ranging from the system level down to that of logic gates, for design entry, documentation, and verification purposes. Since 1987, VHDL has been standardized by the Institute of Electrical and Electronics Engineers (IEEE) as IEEE Std 1076; the latest version of which is IEEE Std 1076-2019. VHDL can be used for designing hardware and for creating test entities to verify the behavior of that hardware VHDL language used to design hardware systems such as FPGA and is an alternative to Verilog. It stands for Very High-Speed IC Description Language.

3.4.3 VHDL Program

IEEE-standardized HDL based on VHDL called VHDL-AMS (officially IEEE 1076.1) has been developed. VHDL is generally used to write text models that describe a logic circuit. Such a model is processed by a synthesis program, only if it is part of the logic design. A simulation program is used to test the logic design using simulation models to represent the logic circuits that interface to the design. This collection of simulation models is commonly called a testbench. VHDL can be used for designing hardware and for creating test entities to verify the behavior of that hardware. VHDL is used as a design entry format by a variety of EDA

tools, Including synthesis tools such as Quartus Prime Integrated Synthesis, simulation tools, and formal verification tools. VHDL language used to design hardware systems such as FPGA and is an alternative to Verilog. It stands for Very High Speed IC Description Language. VHDL has finer control and can be used to design low level systems like gates to high level systems like in Verilog. VHDL is a strongly typed language and is not case sensitive.

A program in VHDL is known as a VHDL model and each VHDL model has two components:

- Entity
- Architecture

In simple terms, Entity can be though of as a class while architecture is a function.

Entity represents the interface specification of the component in the model and the architecture represents the internal structure or implementation of the entity. A design entity is a basic element in a VHDL model. It can have different levels of abstraction like gate, printed circuit board or the entire system.

The different levels of abstraction is classified into:

- Behavioral
- Dataflow
- Structural

3.4.4 Behavior Model

In this modeling style, the behavior of an entity as set of statements is executed sequentially in the specified order. Only statements placed inside a PROCESS, FUNCTION, or PROCEDURE are sequential. PROCESSES, FUNCTIONS, and PROCEDURES are the only sections of code that are executed sequentially. However, as a whole, any of these blocks is still concurrent with any other statements placed outside it.

To model analog and mixed-signal systems, an IEEE-standardized HDL based on VHDL called VHDL-AMS (officially IEEE 1076.1) has been developed. VHDL is generally used to write text models that describe a logic circuit. Such a model is processed by a synthesis program, only if it is part of the logic design. A simulation program is used to test the logic design using simulation models to represent the logic circuits that interface to the design. This collection of simulation models is commonly called a testbench. One important aspect of

behavior code is that it is not limited to sequential logic. Indeed, with it, we can build sequential circuits as well as combinational circuits. The behavior statements are IF, WAIT, CASE, and LOOP. VARIABLES are also restricted and they are supposed to be used in sequential code only. VARIABLE can never be global, so its value cannot be passed out directly. The behavior statements are IF, WAIT, CASE, and LOOP. VARIABLES are also restricted and they are supposed to be used in sequential code only. VARIABLE can never be global, so its value cannot be passed out directly.

- Functional performance is the goal of behavioral modeling
- Timing optionally included in the model
- Software engineering practices should be used to develop behavioral model
- Sequential, inside a process
- Just like a sequential program
- The main character is ‘process (sensitivity list)"

3.4.5 VHDL IF Statement

The if statement is a conditional statement which uses boolean conditions to determine which blocks of VHDL code to execute. Whenever a given condition evaluates as true, the code branch associated with that condition is executed. This statement is similar to conditional statements used in other programming languages. We have seen this in previous posts where we used the if statement to detect a rising edge on a clock signal. This is a very typical use case which we use to model flip flop circuits. It is also possible to include as many elsif branches as necessary to properly model the underlying circuit.

The if statement uses Boolean conditions to determine which lines of code to execute. In the snippet above, these expressions are given by <expression1> and

<expression2>.These expressions are sequentially evaluated and the code associated with a branch is executed when an expression evaluates as true.

Only one branch of an if statement can ever be executed. This is normally the first expression which evaluates as true. The only exception to this occurs when none of the expressions are true. In this instance, the code in the else branch will execute.

If no else branch is associated with the code, then none of the code branches will be executed. The code associated with each branch can include any valid VHDL code, including further if

statements. This approach is known as nested if statements. When using this type of code, we should take care to limit the number of nested statements as it can lead to difficulties in meeting timing.

3.4.6 VHDL Case Statement

We use the VHDL case statement to select a block of code to execute based on the value of a signal. When we write a case statement in VHDL we specify an input signal to monitor and evaluate.

It is possible to exclude the others branch of the statement, although this is not advisable. If the others branch is excluded then all valid values of the <control_signal> must have its own branch. This includes values other than 0 or 1 when using a type such as std_logic or std_logic_vector.

As with the if statement, the code associated with each branch can include any valid VHDL code. This includes further sequential statements, such as if or case statements. Again, we should try to limit the number of nested statements as it makes it easier to meet our timing requirements.

3.5 VHDL DESIGN STYLES

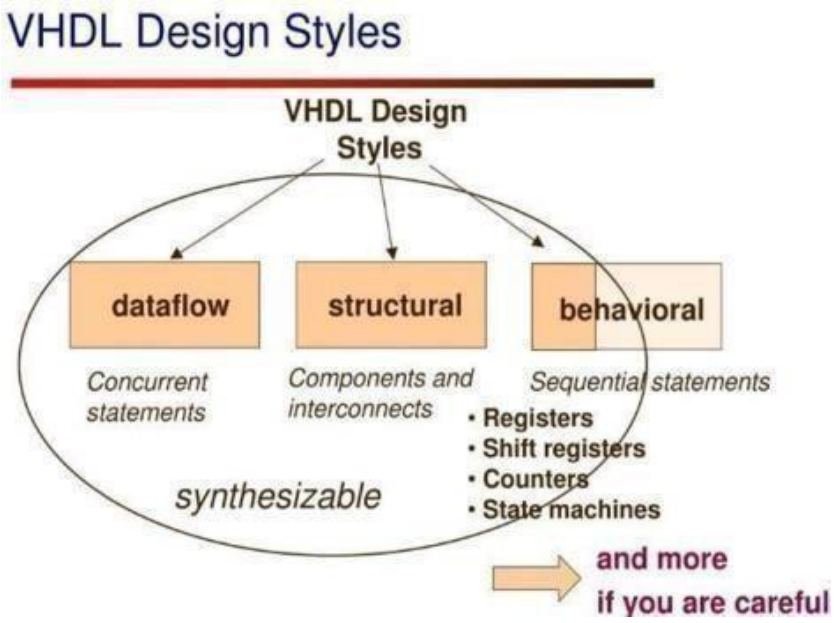


FIGURE 3.4 VHDL DESIGN STYLES

3.5.1 Data Flow Modeling

In data flow modeling style, digital circuits are modeled as a network of interconnected components. The components are connected by signals, which represent the flow of data.In this style, the behavior of a circuit is described by a set of concurrent assignments that define the relationships between the inputs and outputs of the circuit. The concurrent assignments are made up of simple mathematical equations, conditionals, and case statements that define the behavior of the components.

The components are described by their functional behavior, and the interconnections between them are defined by the signal connections. The flow of data is defined by the concurrent assignments, which describe how the inputs and outputs of the components are connected .The data flow modeling style is particularly useful for describing combinatorial logic, where the output depends only on the current input values. In this style, the behavior of the circuit is defined by a set of concurrent assignments that describe how the inputs and outputs are related .One advantage of data flow modeling style is that it is easy to understand and describe the behavior of a circuit. It is also easy to modify and simulate the behavior of the circuit.

However, data flow modeling style can be less effective for describing more complex circuits, such as those with sequential logic or memory. In these cases, other modeling styles, such as behavioral or structural modeling, may be more appropriate.

3.5.2 Structural Modeling

In this modeling, an entity is described as a set of interconnected components. A component instantiation statement is a concurrent statement. Therefore, the order of these statements is not important. The structural style of modeling describes only an interconnection of components (viewed as black boxes), without implying any behavior of the components themselves nor of the entity that they collectively represent.

In Structural modeling, architecture body is composed of two parts – the declarative part (before the keyword begin) and the statement part (after the keyword begin).

Structural model represents the framework for the system and this framework is the place where all other components exist. Hence, the class diagram, component diagram and deployment diagrams are part of structural modeling. They all represent the elements and the mechanism to assemble them.

The structural model never describes the dynamic behavior of the system. Class diagram is the most widely used structural diagram. Structural model represents the framework for the system and this framework is the place where all other components exist. Hence, the class diagram, component diagram and deployment diagrams are part of structural modeling. They all represent the elements and the mechanism to assemble them.

3.6 VIVADO PROJECT CREATING PROCESS

3.6.1 Create a VIVADO Project

VIVADO “projects” are directory structures that contain all the files needed by a particular design. Some of these files are user-created source files that describe and constrain the design, but many others are system files created by VIVADO to manage the design, simulation, and implementation of projects. In a typical design, you will only be concerned with the user-created source files.

But, in the future, if you need more information about your design, or if you need more precise control over certain implementation details, you can access the other files as well. When setting up a project in VIVADO, you must give the project a unique name, choose a location to store all the project files, specify the type of project you are creating, add any pre-existing source files or constraints files (you might add existing sources if you are modifying an earlier design, but if you are creating a new design from scratch, you won’t add any existing files – you haven’t written them yet), and finally, select which physical chip you are designing for. These steps are illustrated below.

3.6.2 Start Vivado

Figure 3.5: VIVADO home page

In Windows, you can start VIVADO by clicking the shortcut on the desktop. After VIVADO is started, the window should look similar to the picture in Fig

3.6.3 Open Create Project Dialog

In the home page you can observe a quick start, in the quick start you have to click on the create project which is look similar to the Fig 3.6.3

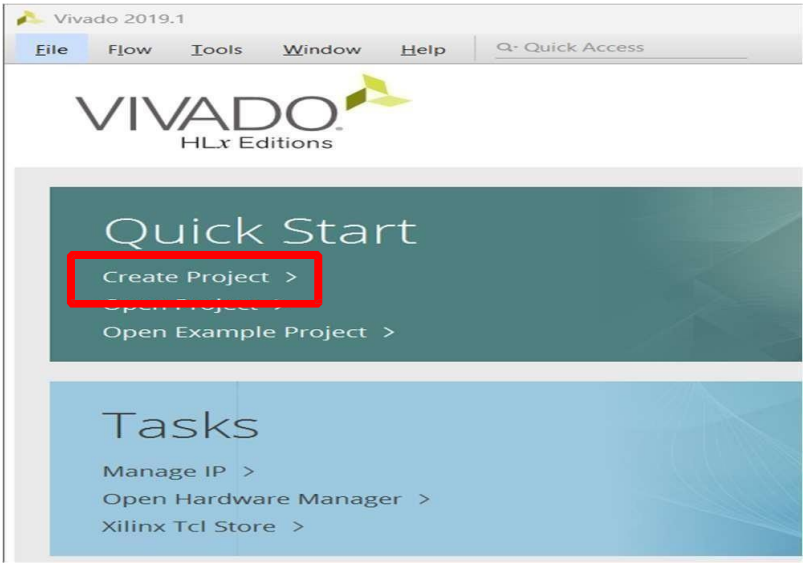


Figure 3.6: Create project set up

Click on “Create Project”, This will open the New Project dialog as shown in the figure 3.6.3.2. click on the next to continue

3.7 Set Project Name And Location

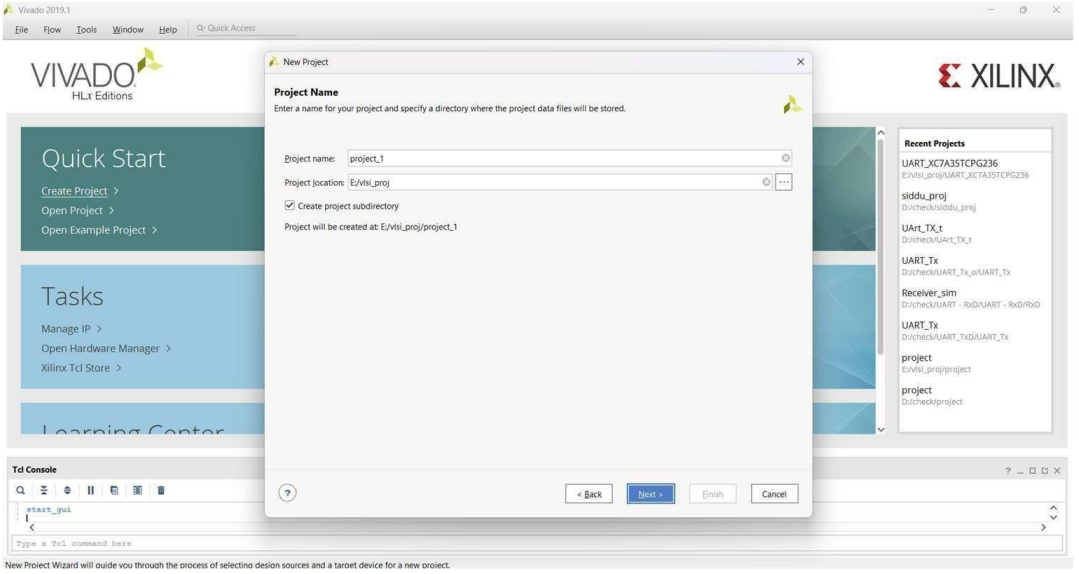


Figure 3.7: Enter project name

Enter a name for the project. In the figure, the project name is “project_1”, which isn’t a particularly useful name. It’s usually a good idea to make the project name more descriptive, so you can more readily identify your designs in the future.

3.8 Select Project Type

The “project type” configures certain design tools and the IDE appearance based on the type of project you are intending to create. Most of the time, and for all Real Digital courses, you will choose “RTL Project” to configure the tools for the creation of a new design. (RTL stands for Register Transfer Language, which is a term sometimes used to mean a hardware design language like Verilog).

3.9 Add Existing Sources

In a typical new or early-stage design, you won’t add any existing sources because you haven’t created them yet. But as you complete more designs and build up a library of previously completed and known good designs, you may elect to add sources and them use them in a new design. For now, there are no existing sources to add, so just click Next.

3.10 Add Constraints

Constraint files provide information about the physical implementation of the design. They are created by the user, and used by

the synthesizer. Constraints are parameters that specify certain details about the design. As examples, some constraints identify which physical pins on the chip are to be connected to which named circuit nodes in your design; some constraints setup various physical attributes of the chip, like I/O pin drive strength (high or low current); and some constraints identify physical locations of certain circuit components. The Xilinx Design Constraints (.xdc filetype) is the file format used for describing design constraints, and you need to create an .xdc file in order to synthesize your designs for a Real Digital board. Later in this tutorial, you will create a constraints file to identify which named circuit nodes must be connected to which physical pins. But for now, you have no existing constraints file to add, so you can simply click next.

3.11 Select Parts

Xilinx produces many different parts, and the synthesizer needs to know exactly what part you are using so it can produce the correct programming file. To specify the correct part, you need to know the device family and package, and less critically, the speed and temperature grades (the speed and temperature grades only affect special- purpose simulation results, and they have no effect on the synthesizer’s ability to produce accurate circuits). You must choose the appropriate part for the device installed on your board.

For example, the project uses a basys3 device with the following attributes:

Category : All

Family : Artix-7

Package : cpg236

Speed : -1

Select the Xilinx board for the project as xc7a35cpg236-1 which can have the 20800 LUT elements, 41600 Flipflops, 50 Block RAMs, 90 DSPs. Click on next

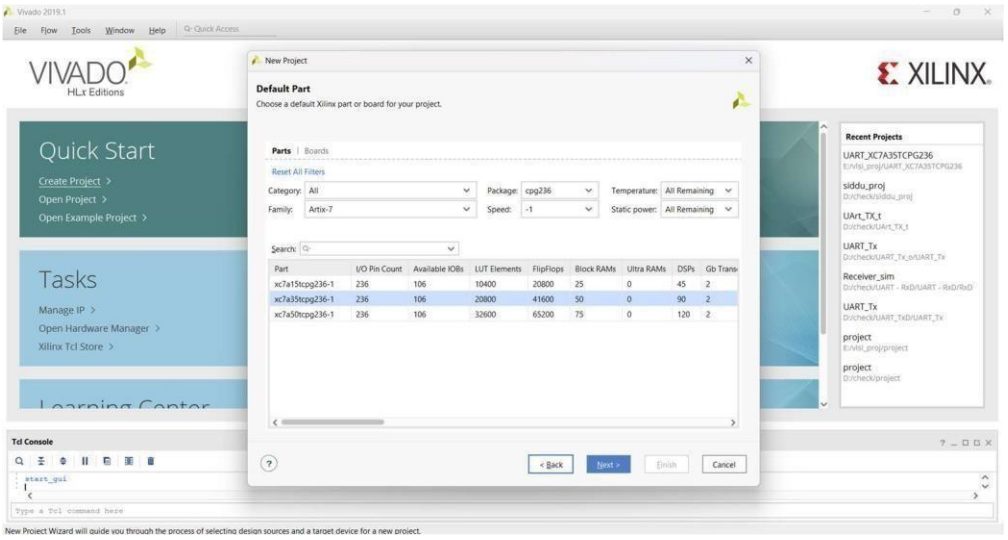


Figure 3.8: Select Artix-7 CPG236 board

3.12 Create Project Configuration Summary

On the last page of the Create Project Wizard a summary of the project configuration is shown. Verify all the information in the summary is correct, and in particular make sure the correct FPGA part is selected. If anything is incorrect, click back and fix it; otherwise, click Finish to finish creating an empty project.

3.13 Vivado Project Window

After you have finished with the Create Project Wizard, the main IDE window will be displayed. This is the main “working” window where you enter and simulate your Verilog code, launch the synthesizer, and program your board. The left-most pane is the flow navigator that shows all the current files in the project, and the processes you can run on those files. To the right of the flow navigator is the project manager window where you enter source code, view simulation data, and interact with your design. The console window across the bottom shows a running status log. Over the next few projects, you will interact with all of the panels.

3.14 Edit the Project - Create Source Files

All projects require at least two types of source files – an HDL file (Verilog or VHDL) to describe the circuit, and a constraints file to provide the synthesizer with the information it needs to map your circuit into the target chip. This tutorial presents the steps required to implement a Verilog circuit on your Real Digital board: first, a Verilog source file is created to define the circuits behavior (again, for this tutorial, you can simply copy or download the completed file rather than typing it); second, a constraints files is created to define how the Verilog circuit is mapped into the Xilinx logic device (again, copied or downloaded for this tutorial); third, the Verilog source file and constraints file are synthesized into a ".bit" file that can be programmed onto your board; and fourth, the device is configured with the circuit.

After the Verilog source file is created, it can be directly simulated. Simulation (discussed in more detail later) lets you work with a computer model of a circuit, so you can check its behavior before taking the time to implement it in a physical device. The simulator lets you drive all the circuit inputs with varying patterns over time, and to check that the outputs behave as expected under all conditions.

After the constraint file is created, the design can be synthesized. The synthesis process translates Verilog source code into logical operations, and it uses the constraints file to map the logical operations into a given chip. In particular (for our needs here), the constraints file defines which Verilog circuit nodes are attached to which pins on the Xilinx chip package, and therefore, which circuit nodes are attached to which physical devices on your board. The synthesis process creates a ".bit" file that can be directly programmed into the Xilinx chip.

In this first tutorial, the Verilog and constraint source files are provided for you. Instead of creating them yourself as would normally be the case, you can simply copy them into empty source files, or download them and include them in your project directly. In later designs, you will create these files yourself.

3.15 Design Source

There are many ways to define a logic circuit, and many types of source files including VHDL, Verilog, EDIF and NGC netlists, DCP checkpoint files, TCL scripts, System C files, and many others. We will use the Verilog language in this course, and introduce it gradually over the first several projects. For now, you can get familiar with some of the basic concepts by reading the following.

To create a VHDL source file for your project, right-click on “Design Sources” in the Sources panel, and select Add Sources. The Add Sources dialog box will appear as shown – select “Add or create design sources” and click next.

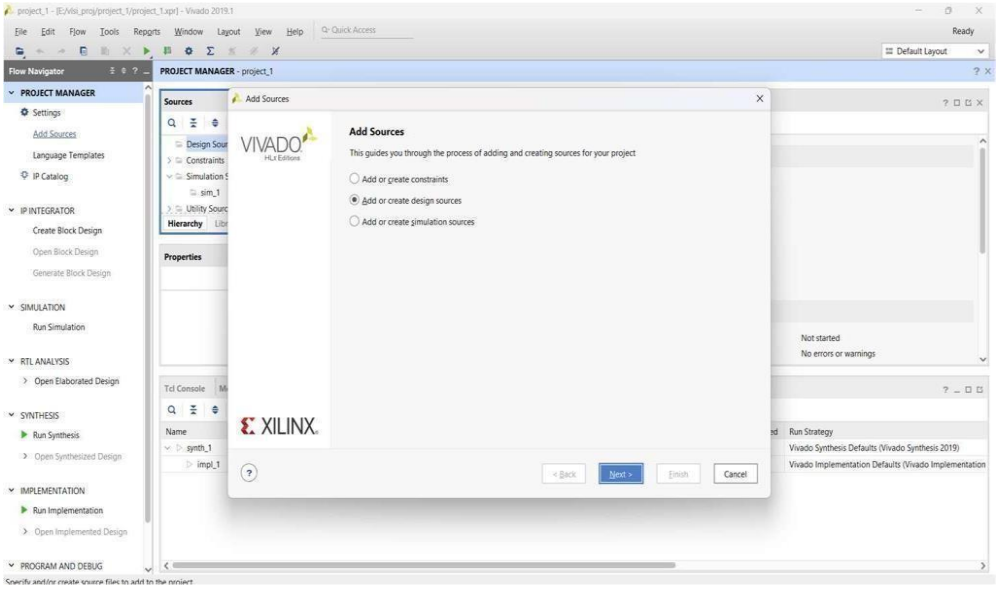


Figure 3.9: Add or create design sources window

In the Add or Create Design Sources dialog, click on Create File, enter “project1_demo” as filename, and click OK. The newly created file will appear in the list as shown. Click Finish to move to the next step. Skip the Define Module dialog by clicking OK to continue. You will now see project1_demo listed underneath Design Sources folder in the Sources panel.

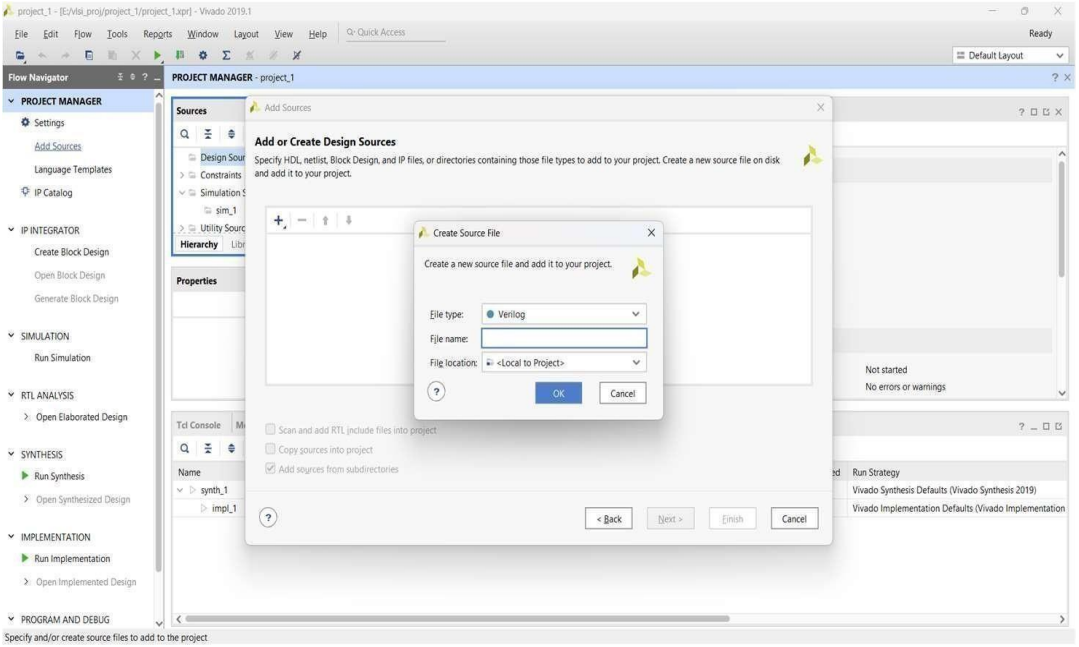


Figure 3.10: Create design source file

Double click project1_demo to open the file, and replace the contents (copy and paste) with the code below.

Alternative: Download and Add Source

Instead of creating a source file by using copy and paste as described above, you can alternatively download the Project1_demo. V file and add it to your project using the Add Sources button in Add Sources dialog.

3.16 Design Constraints

Design sources, like VHDL files, only describe circuit behavior. You must also provide a constraints file to map your design into the physical chip and board you are working with.

To create a constraint file, expand the Constraints heading in the Sources panel, right- click on constrs_1, and select Add Sources.

An Add Sources dialog will appear as shown. Select Add or Create Constraints and click Next to cause the “Add or Create Constraints” dialog box to appear.

Click on Create File, enter project1 for the filename and click OK. The newly created file will appear in the list as shown.

Click Finish to move to the next step. You will now see project1.xdc listed underneath Constraints/constrs_1 folder in the Sources panel. Double click project1.xdc to open the file, and replace the contents with the code.

CHAPTER 4
WORKING OF THE SYSTEM

4.1 Image Capture

The first stage of the system is image acquisition. The webcam is mounted on the ship in such a way that it captures the forward view of the surroundings. It continuously captures video frames at a fixed resolution (for example, 640×480 pixels).

These frames are transmitted to the FPGA board for further processing. Before applying object detection, basic preprocessing steps such as noise reduction, normalization, and resizing may be performed to improve the quality of the input image. This ensures that the detection algorithm works efficiently even under varying environmental conditions.

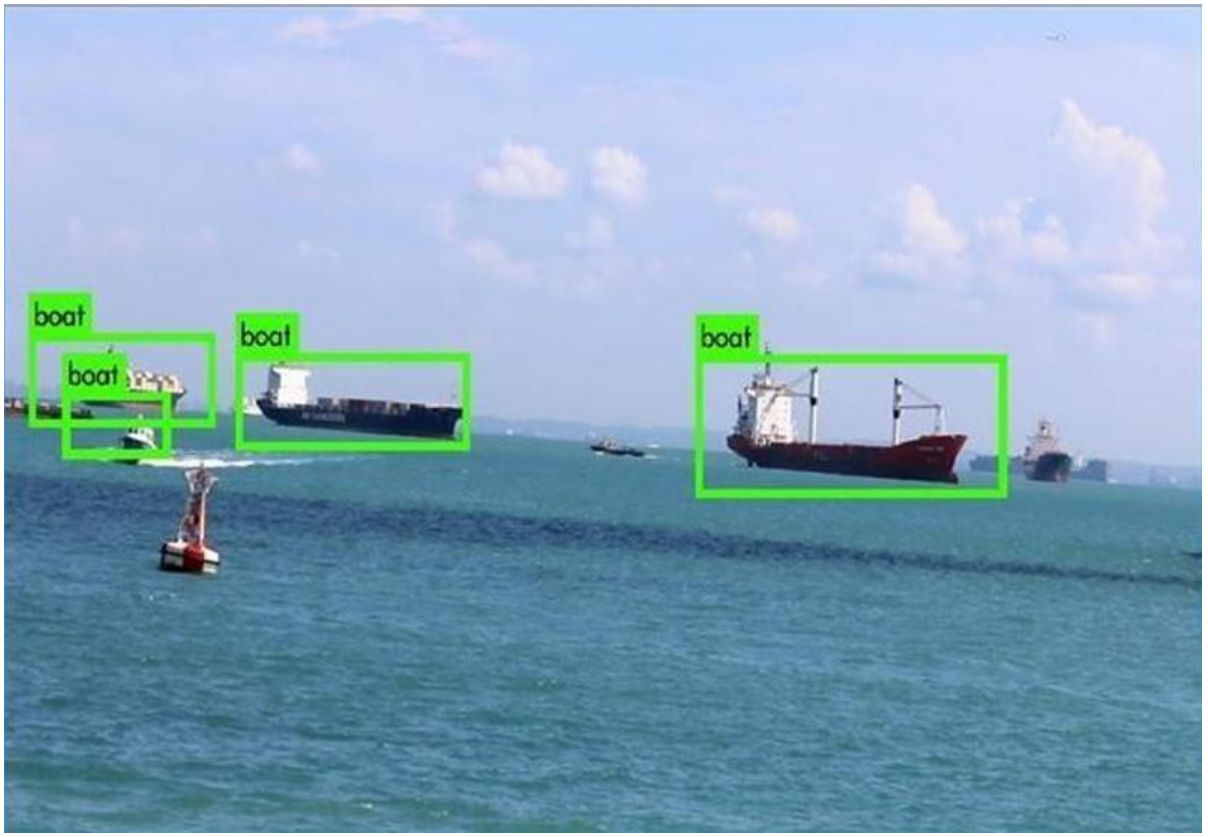


FIGURE 4.1: IMAGE CAPTURE

4.2 Object Detection

After preprocessing, the captured frames are passed to the YOLOv3-tiny object detection model implemented on the FPGA platform. This model is designed for fast and efficient real-time detection.

The algorithm divides the image into grids and predicts bounding boxes along with class probabilities. It identifies objects such as ships, buoys, debris, and other obstacles present in the water. Each detected object is assigned a confidence score indicating the accuracy of detection. Due to FPGA's parallel processing capability, multiple computations are performed simultaneously, resulting in low latency (around 1.5 seconds). This allows the system to respond quickly to dynamic changes in the environment.

FIGURE 4.2: OBJECT DETECTION

Object detection and lane detection are necessary for preventing collision and maintaining the ship on course. With advanced sensors like LIDAR, radar, and cameras, autonomous ships are capable of real-time detection of obstacles and can respond with fast decisions to prevent collision. This will be particularly significant in congested shipping channels or areas of limited visibility. Benefit:

Substantially minimizes the risk of accidents due to human error or reduced visibility, and it increases the overall safety of the ship and cargo

4.3 Distance Measurement

While the camera handles long-range detection, ultrasonic sensors are used for short-range distance measurement. These sensors emit ultrasonic waves and measure the time taken for the echo to return after hitting an object.Based on this time delay, the distance to the obstacle is calculated. This helps in detecting objects that are very close to the ship and may not be clearly visible in the camera feed. The combination of camera-based detection and ultrasonic sensing improves the overall accuracy and reliability of obstacle detection, especially in complex scenarios.

FIG 4.3: DISTANCE MEASUREMENT

4.4 Route Planning

The route planning stage uses the A* (A-Star) algorithm to determine the optimal path from the current position to the destination.

The algorithm considers multiple parameters such as distance, detected obstacles, and environmental conditions while calculating the path. It selects the shortest and safest route by avoiding obstacles and minimizing travel cost.

The path is dynamically updated whenever new obstacle data is received, ensuring real-time adaptability of the system.

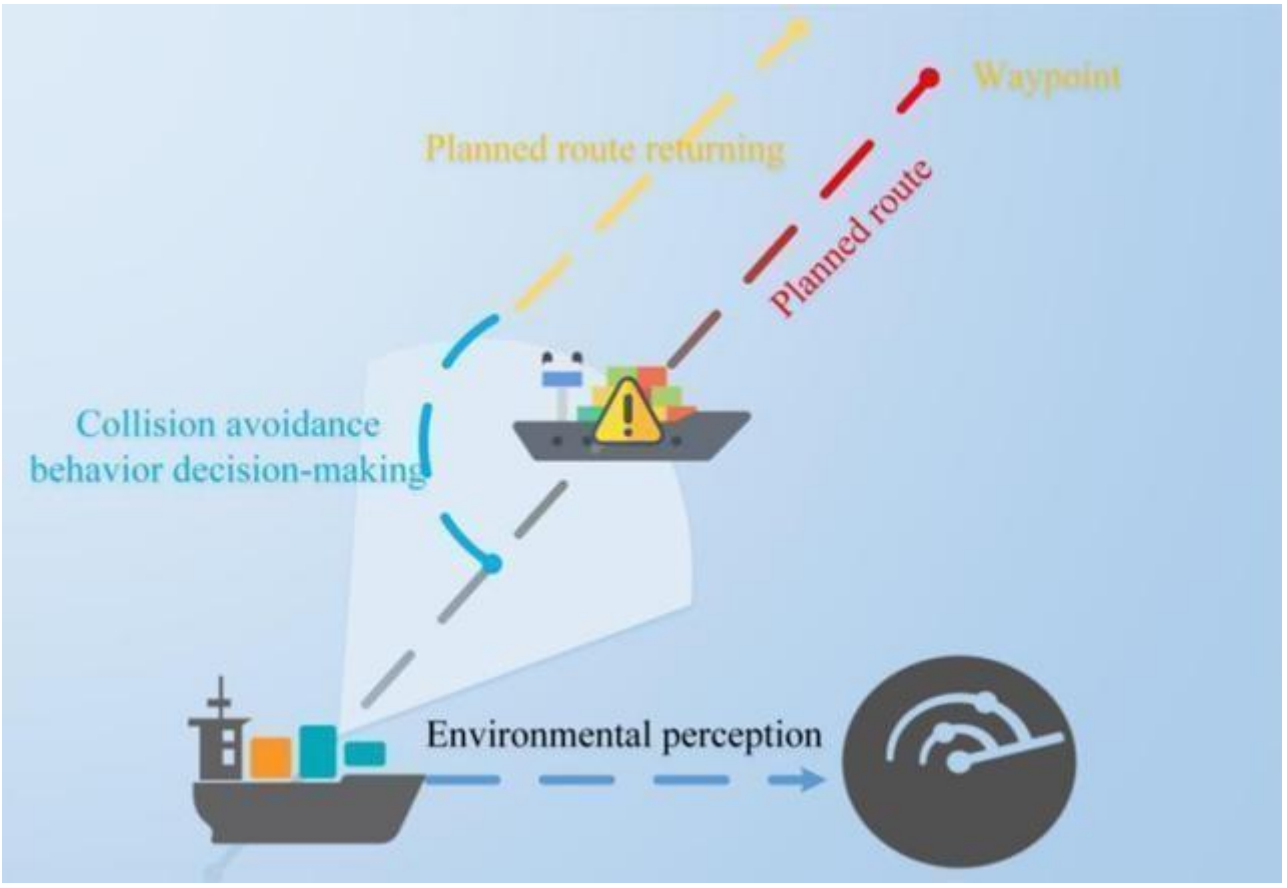


FIGURE 4.4: ROUTE PLANNING

4.5 Motor Control

Based on the output of the route planning stage, the FPGA generates appropriate control signalsto move the ship. The FPGA sends PWM (Pulse Width Modulation) signals through GPIO pins to the BTS7960 motor driver. The motor driver then controls the servo motors, enabling different movements such as forward, backward, left turn, right turn, and stop.

This allows precise control of the ship's direction and speed, ensuring smooth navigation according to the planned path.

FIGURE 4.5: MOTOR CONTROL

4.6 REALVNC VIEWER

REALVNC VIEWER is a remote desktop application that allows users to access and control another computer from a different location. It works using Virtual Network Computing (VNC) technology, which transmits the screen of one device to another over a network.

It is widely used to remotely control systems like Raspberry Pi, especially in IoT and embedded projects.

Working Principle

RealVNC follows a **client-server architecture**:

- **VNC Server** runs on the device to be controlled (e.g., Raspberry Pi)
- **VNC Viewer** runs on the user's device (laptop/PC/mobile)

Steps:

1. The VNC Server continuously captures the screen (frame buffer) of the remote device.
2. The captured screen data is compressed and transmitted over the network.
3. The VNC Viewer receives this data and displays the remote desktop interface in real time.
4. User inputs such as keyboard strokes and mouse movements are sent back to the VNC Server.
5. The server processes these inputs and updates the remote system accordingly.

Key Features

- Remote access from anywhere
- Cross-platform support (Windows, Linux, Android, iOS)

- Secure connection with encryption
- File transfer support
- Easy setup and user-friendly interface

RealVNC is an efficient and reliable tool for remote system access and control. In embedded and IoT projects, it plays a crucial role in managing devices such as the Raspberry Pi 4 Model B remotely, making both development and monitoring processes much easier and more flexible.

RealVNC Viewer significantly simplifies system management in embedded applications by reducing the dependency on external hardware components like monitors, keyboards, and mice. It improves accessibility by allowing developers to connect to the system from anywhere through a network. This remote capability is especially useful during testing, debugging, and deployment stages, where direct physical access to the device may not always be possible.

Furthermore, it enables real-time interaction with the system, allowing users to execute programs, monitor outputs, and modify configurations instantly. This enhances productivity and reduces the time required for troubleshooting and maintenance. The secure communication provided by encryption ensures that data transmission between the client and server remains safe and protected from unauthorized access.

In advanced applications such as IoT systems, robotics, and autonomous navigation, RealVNC Viewer provides a convenient interface for continuous system monitoring and control. In the proposed autonomous ship project, it plays a vital role in remotely accessing the Raspberry Pi and edge devices, enabling developers to observe system performance, view camera outputs, and manage operations without being physically present.

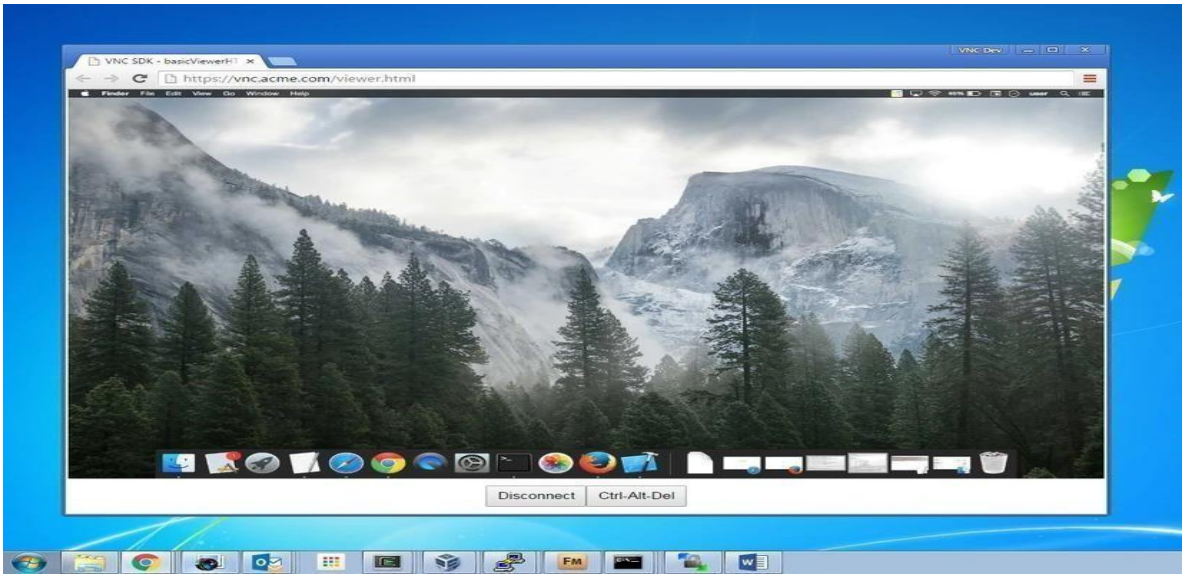


FIGURE 4.6: REALVNC VIEWER

4.6.1. *Client System (User Side)*

- This is the system where the user opens a browser.
- The browser accesses a URL (shown as vnc.html).
- Acts as the **VNC Viewer**.

Functions:

- Displays the remote desktop
- Sends user inputs (keyboard, mouse) to the remote system

4.6.2. *Web Browser Interface*

- Runs a **web-based VNC client (like noVNC)**.

Functions:

- Provides GUI for remote access
- Works without installing software
- Converts browser inputs into VNC protocol messages.

4.6.3. *VNC Server (Remote System)*

- Installed on the remote machine (Mac OS desktop shown).
- Captures screen and sends it to the client.

Functions:

- Shares the desktop screen
- Executes user commands remotely

4.6.4. *Network (Internet / LAN)*

- Connects client and server.
- Uses protocols like:
    - TCP/IP
    - WebSockets (for browser-based VNC)

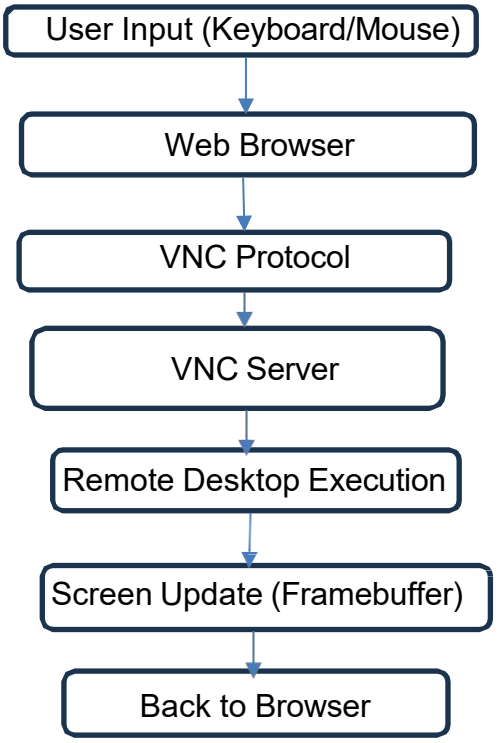
4.6.5. *Remote Desktop (Host Machine)*

- The actual system being controlled.
- In your image, it shows a **Mac OS desktop**.

Functions:

- Runs applications
- Responds to user input from client

4.6.6 *FLOW CHART*



User Input (Keyboard / Mouse)

- You (user) interact with your system:
  - Move mouse
  - Type on keyboard
- These actions are captured by the browser.

Example: Clicking an icon or typing a command

Web Browser

- The browser (with VNC web client like noVNC) receives your input.
- It converts your actions into digital signals.

Role:

- Acts as an interface between user and remote system

VNC Protocol

- Your inputs are encoded into **VNC protocol messages**.
- Sent over the network using:

- TCP/IP
- WebSockets (for browser-based VNC)

Role:

- Ensures proper communication between client and server

VNC Server (Remote Machine)

- Receives the input signals.
- Interprets them as real user actions.

Example:

- Mouse click → Opens application
- Keyboard input → Types on remote system

Remote Desktop Execution

- The remote computer processes your commands.
- Applications run just like you are physically present.

Example

- Opening files
- Running software
- Navigating OS

Screen Update (Framebuffer)

- The VNC server captures the updated screen.
- This is called a **framebuffer** (image of the screen).

Role:

- Continuously captures display changes

Back to Browser

- The updated screen is sent back to your browser.
- You see real-time updates.

Result:

- You feel like you are directly using the remote computer

CHAPTER 5
CONCLUSION AND FUTURE SCOPE

CONCLUSION:

The FPGA-based autonomous ship system successfully demonstrates an efficient approach for real-time navigation and obstacle detection by integrating sensor fusion and machine learning techniques. The developed prototype is capable of sensing its environment, identifying obstacles, and making navigation decisions without human intervention, thereby reducing the chances of human error in maritime operations.

By combining camera-based vision using the YOLOv3-tiny algorithm, ultrasonic sensors for short-range detection, and GPS-based route planning using the A* algorithm, the system achieves reliable and accurate navigation. The integration of these technologies ensures that the ship can detect both distant and nearby obstacles and respond accordingly in real time.

One of the key advantages of this system is the use of the FPGA platform, which enables parallel processing of multiple tasks such as image processing, object detection, and path planning. This results in faster computation, reduced latency, and significantly lower power consumption compared to traditional CPU or GPU-based systems. These features make the system highly suitable for embedded and energy-efficient maritime applications.

The incorporation of cloud connectivity through Azure IoT Hub further enhances the system capabilities by enabling real-time data transmission, remote monitoring, and predictive maintenance. This allows continuous tracking of the ship's performance and improves operational efficiency.

Overall, the proposed system provides a scalable, low-power, and intelligent solution for autonomous maritime navigation. It lays a strong foundation for the development of advanced autonomous ships, smart ports, and future maritime transportation systems that are safer, more efficient, and environmentally sustainable.

FUTURE SCOPE:

The proposed FPGA-based autonomous ship system provides a strong foundation for intelligent maritime navigation. However, there are several areas where the system can be further enhanced to improve performance, scalability, and real-world applicability.

In the future, more advanced deep learning models can be implemented to improve object detection accuracy. Models like YOLOv5 or YOLOv8 can be optimized and deployed on FPGA platforms to achieve better precision, especially in challenging conditions such as low lighting, fog, or heavy rain.

The system can also be enhanced by integrating additional sensors such as LiDAR, radar, and infrared cameras. These sensors can provide more detailed environmental data and improve detection reliability in complex maritime environments. Sensor fusion techniques can be further improved to combine data from multiple sources for better decision-making.

Another important enhancement is the inclusion of real-time weather and ocean data such as wind speed, wave height, and tide conditions. By incorporating these parameters into the path planning algorithm, the system can generate safer and more efficient navigation routes.

The project can be extended to support communication between multiple autonomous ships using technologies like V2V (Vessel-to-Vessel communication). This would enable coordinated navigation, collision avoidance, and better traffic management in busy waterways and ports.

Cloud integration can be further expanded by using advanced data analytics and machine learning techniques for predictive maintenance and performance optimization. Real-time dashboards and alerts can be developed to monitor fleet operations more effectively.

In addition, the system can be scaled and tested in real-world maritime environments such as rivers, ports, and open seas. This would help validate the system’s reliability and performance under different conditions and improve its practical implementation.

Finally, the development of fully autonomous shipping systems can contribute to smart ports and sustainable maritime transportation by reducing fuel consumption, minimizing human error, and lowering environmental impact.

REFERENCES

[1]. Raspberry Pi 4 Model B Documentation, Raspberry Pi Foundation, Available: Official Raspberry Pi Website.

[2]. NVIDIA Jetson Nano Developer Kit User Guide, NVIDIA Corporation, Available: NVIDIA Developer Documentation.

[3]. Raspberry Pi Camera Module V2 Documentation, Raspberry Pi Foundation.

[4]. Joseph Redmon et al., “You Only Look Once: Unified, Real-Time Object Detection,” IEEE
a. CVPR, 2016.

[5]. Ultralytics, YOLOv5/YOLOv8 Documentation, Available: Ultralytics Official Website.

[6]. OpenCV Library Documentation, Available: OpenCV Official Website.

[7]. Hart, Nilsson, Raphael, “A Formal Basis for the A* Pathfinding Algorithm,” IEEE
a. Transactions, 1968.

[8]. Flask Documentation, Available: Flask Official Website.

[9]. Django Documentation, Available: Django Official Website.

[10]. Node.js Documentation, Available: Node.js Official Website.

[11]. HTML5 and CSS3 Web Standards Documentation, W3C.

[12]. JavaScript Programming Guide, Available: Mozilla Developer Network (MDN).

[13]. IEEE Xplore Digital Library, Research Papers on Autonomous Systems and Edge Computing.

[14]. Embedded Systems Design using Raspberry Pi and Edge AI, NVIDIA & Raspberry Pi Developer Resources.

[15]. Autonomous Navigation Systems Research Articles, ScienceDirect & Springer Journal

APPENDIX

APPENDIX FOR EDGE BOARD:

```
module
mot( input clk,
input rst_n,
input echo1,       // Front ultrasonic echo
input echo2,       // Back ultrasonic echo
output reg trig1,  // Front ultrasonic trigger
output reg trig2,  // Back ultrasonic trigger
output reg pwm_servo, // PWM to servo motor
output reg [5:0] motor, // Motor control outputs [IN1, IN2, ENA, IN3, IN4, ENB]
output reg LED1,  // Obstacle front
output reg LED2,  // Motor control outputs [IN1, IN2, ENA, IN3, IN4, ENB]
output reg Led1,  // Obstacle front
output reg Led2     // Obstacle back
);

// ----- Parameters -----
parameter CLOCK_FREQ = 50000000;
parameter PWM_FREQ  = 50;
parameter PWM_PERIOD = CLOCK_FREQ / PWM_FREQ;
parameter MIN_PULSE  = CLOCK_FREQ / 1000;  // 1ms (0°)
parameter MAX_PULSE  = CLOCK_FREQ / 500;     // 2ms (180°)
parameter DIST_THRESH = 40;
parameter TRIG_PULSE_WIDTH = 500;

// ----- Servo rotation FSM -----
reg [1:0] servo_state = 0;
reg [31:0] servo_counter = 0;
reg [7:0] angle = 90;
reg [31:0] pulse_width;

always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        servo_state <= 0;
        servo_counter <= 0;
        angle <= 90;
    end else begin
        if (servo_counter >= 25_000_000) begin
            servo_counter <= 0;
            servo_state <= servo_state + 1;

            case (servo_state)
                2'd0: angle <= 135; // Right
                2'd1: angle <= 90;  // Center
                2'd2: angle <= 45;  // Left
                2'd3: angle <= 90;  // Back to Center
            endcase
```

```
        end else

                servo_counter <= servo_counter + 1;
    end
end

// ----- PWM generation for Servo -----
always @(*) begin
    pulse_width = MIN_PULSE + ((MAX_PULSE - MIN_PULSE) * angle) / 180;
end

reg [31:0] pwm_counter = 0;
always @(posedge clk) begin
    if (pwm_counter < pulse_width)
        pwm_servo <= 1;
    else
        pwm_servo <= 0;

    if (pwm_counter >= PWM_PERIOD)
        pwm_counter <= 0;
    else
        pwm_counter <= pwm_counter + 1;
end

// ----- Ultrasonic Trigger Logic -----
reg [19:0] trig_timer1 = 0;
reg [19:0] trig_timer2 = 0;

always @(posedge clk) begin
    trig_timer1 <= (trig_timer1 >= 1000000) ? 0 : trig_timer1 + 1;
    trig1 <= (trig_timer1 < TRIG_PULSE_WIDTH) ? 1 : 0;

    trig_timer2 <= (trig_timer2 >= 1000000) ? 0 : trig_timer2 + 1;
    trig2 <= (trig_timer2 < TRIG_PULSE_WIDTH) ? 1 : 0;
end

// ----- Echo Measurement -----
reg [31:0] counter_echo1 = 0, counter_echo2 = 0;
reg [31:0] distance1 = 0, distance2 = 0;
reg prev_echo1 = 0, prev_echo2 = 0;

always @(posedge clk) begin
    // Front echo
    prev_echo1 <= echo1;
    if (echo1 && !prev_echo1)
        counter_echo1 <= 0;
    else if (echo1)
        counter_echo1 <= counter_echo1 + 1;
    else if (!echo1 && prev_echo1)
        distance1 <= counter_echo1 / 2900;
```

```
    // Back echo
    prev_echo2 <= echo2;

            if (echo2 && !prev_echo2)
        counter_echo2 <= 0;
    else if (echo2)
        counter_echo2 <= counter_echo2 + 1;
    else if (!echo2 && prev_echo2)
        distance2 <= counter_echo2 / 2900;
end

// ----- LED indication -----
reg [23:0] led1_timer = 0, led2_timer = 0;
always @(posedge clk) begin
    if (distance1 > 0 && distance1 < DIST_THRESH)
        led1_timer <= 24'd5000000;
    else if (led1_timer > 0)
        led1_timer <= led1_timer - 1;
    LED1 <= (led1_timer > 0);
    Led1 <= (led1_timer > 0);

    if (distance2 > 0 && distance2 < DIST_THRESH)
        led2_timer <= 24'd5000000;
    else if (led2_timer > 0)
        led2_timer <= led2_timer - 1;
    LED2 <= (led2_timer > 0);
    Led2 <= (led2_timer > 0);
end

// ----- Motor Control -----
reg [31:0] pwm_counter_motor = 0;
reg pwm_motor;

wire [31:0] full_speed = 32'd750000; // Full speed PWM
wire [31:0] slow_speed = 32'd250000; // Slow speed when obstacle detected
reg [31:0] selected_speed;

always @(*) begin
    if (distance1 < DIST_THRESH && distance1 > 0)
        selected_speed = slow_speed;
    else
        selected_speed = full_speed;
end

always @(posedge clk) begin
    pwm_motor <= (pwm_counter_motor < selected_speed) ? 1 : 0;
    pwm_counter_motor <= (pwm_counter_motor >= 1000000) ? 0 : pwm_counter_motor + 1;
end

// Direction Control
always @(*) begin
```

```
        if (distance1 < DIST_THRESH && distance1 > 0) begin
            // Turn right when obstacle in front
            motor[0] = 1; // IN1

                    motor[1] = 0; //
            IN2 motor[3] = 1; //
            IN3 motor[4] = 0; //
            IN4
        end
        motor[2] = pwm_motor; // ENA
        motor[5] = pwm_motor; // ENB
    end

endmodule
```

APENDEX FOR RASPBERRY PI :

```
import cv2
from ultralytics import YOLO
import threading
import time
import RPi.GPIO as GPIO

# ---------- GPIO SETUP ----------
GPIO.setmode(GPIO.BCM)

FPGA_FRONT = 22  # From FPGA (LED1)
FPGA_BACK  = 23  # From FPGA (LED2)

GPIO.setup(FPGA_FRONT, GPIO.IN, pull_up_down=GPIO.PUD_DOWN)
GPIO.setup(FPGA_BACK,  GPIO.IN, pull_up_down=GPIO.PUD_DOWN)

# Output pins (Pi -> FPGA or motor control)
OUT_FRONT = 17
OUT_LEFT  = 27
OUT_RIGHT = 24

GPIO.setup(OUT_FRONT, GPIO.OUT)
GPIO.setup(OUT_LEFT,  GPIO.OUT)
GPIO.setup(OUT_RIGHT, GPIO.OUT)

# ---------- YOLO SETUP ----------
model = YOLO("prabha.pt")
model.to("cpu")

# – ---------- THREAD CONTROL – ----------
frame = None
running = True
lock = threading.Lock()

# ---------- CAMERA THREAD ----------
```

```
def capture_frames():
    global frame, running
    cap = cv2.VideoCapture(0)

    cap.set(cv2.CAP_PROP_FRAME_WIDTH, 640)
    cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 480)
    cap.set(cv2.CAP_PROP_BUFFERSIZE, 1)

    while running:
        ret, img = cap.read()
        if ret:
            with lock:
                frame = img.copy()

    cap.release()

threading.Thread(target=capture_frames, daemon=True).start()

frame_count = 0

# ---------- MAIN LOOP ----------
try:
    while True:

        # ---- Read FPGA Sensors ----
        fpga_front = GPIO.input(FPGA_FRONT)
        fpga_back  = GPIO.input(FPGA_BACK)

        # ---- Get Frame Safely ----
        with lock:
            if frame is None:
                continue
            img = frame.copy()

        start = time.time()
        frame_count += 1

        # Skip frames (performance)
        if frame_count % 2 != 0:
            continue

        # ---- YOLO Detection ----
        results = model(img, imgsz=256, conf=0.6, verbose=False)

        h, w, _ = img.shape

        left_detect = 0
        right_detect = 0
        center_detect = 0

        annotated = img.copy()
```

```
for box in results[0].boxes:
    conf = float(box.conf[0])
    cls = int(box.cls[0])
    label = model.names[cls]

    if conf > 0.75 and label in ["boat", "person", "ship"]:

        x1, y1, x2, y2 = map(int, box.xyxy[0])
        center_x = (x1 + x2) // 2

        # Draw box
        cv2.rectangle(annotated, (x1, y1), (x2, y2), (0,255,0), 3)
        cv2.putText(annotated, f"{label} {conf:.2f}",
                    (x1, y1-10),
                    cv2.FONT_HERSHEY_SIMPLEX,
                    0.7, (0,255,0), 2)

        # Zone detection
        if center_x < w//3:
            left_detect = 1
        elif center_x > 2*w//3:
            right_detect = 1
        else:
            center_detect = 1

# ---------- DECISION LOGIC ----------
# Priority: YOLO > FPGA

if center_detect:
    decision = "STOP (Vision)"
    GPIO.output(OUT_FRONT, 0)
    GPIO.output(OUT_LEFT, 0)
    GPIO.output(OUT_RIGHT, 0)

elif left_detect:
    decision = "TURN RIGHT (Vision)"
    GPIO.output(OUT_FRONT, 1)
    GPIO.output(OUT_LEFT, 0)
    GPIO.output(OUT_RIGHT, 1)

elif right_detect:
    decision = "TURN LEFT (Vision)"
    GPIO.output(OUT_FRONT, 1)
    GPIO.output(OUT_LEFT, 1)
    GPIO.output(OUT_RIGHT, 0)

elif fpga_front == 1:
    decision = "STOP (Ultrasonic Front)"
    GPIO.output(OUT_FRONT, 0)
    GPIO.output(OUT_LEFT, 0)
    GPIO.output(OUT_RIGHT, 0)
```

```
        elif fpga_back == 1:
            decision = "BACK OBSTACLE (Warning)"
            GPIO.output(OUT_FRONT, 1)

        else:
            decision = "MOVE FORWARD"

                GPIO.output(OUT_FRONT, 1)
            GPIO.output(OUT_LEFT, 0)
            GPIO.output(OUT_RIGHT, 0)

        # ---------- FPS ----------
        fps = 1 / (time.time() - start)

        cv2.putText(annotated, f"{decision}",
                    (10, 60),
                    cv2.FONT_HERSHEY_SIMPLEX,
                    0.8, (0,0,255), 2)

        cv2.putText(annotated, f"FPS: {int(fps)}",
                    (10, 30),
                    cv2.FONT_HERSHEY_SIMPLEX,
                    1, (255,0,0), 2)

        # Debug print
        print(f"FPGA Front={fpga_front}, Back={fpga_back} | Decision={decision}")

        cv2.imshow("Autonomous System", annotated)

        if cv2.waitKey(1) & 0xFF == ord('q'):
            running = False
            break

# ---------- CLEANUP ----------
except KeyboardInterrupt:
    pass

finally:
    GPIO.cleanup()
    cv2.destroyAllWindows()
```

Publication

From: **VDAT 2025** <vdat2025@iitrpr.ac.in>
Date: Wed, 16 Jul, 2025, 1:15 pm
Subject: Reg. Paper ID 136 Registration - Full Registration required
To: Nikhil <leelanikhil2004@gmail.com>

Dear Suvarnaganti Leela Nikhil,

As informed to you earlier and also mentioned in the Registration form, an accepted paper should have at least one full registration (working professionals/academicians).

As this is not met for Paper ID 136, we request you to pay the differential amount (Rs. 4130) to complete the registration of your paper.

Further, please send both the payment proofs (Bank Name and UTR number should be visible) to this email. This is needed for receipt generation.

Thanks & Regards,
VDAT 2025

From: **VDAT 2025 Team** <vdat2025web@gmail.com>
Date: Sun, 3 Aug, 2025, 7:39 pm
Subject: VDAT 2025 Registration Invoice - 0131
To: <leelanikhil2004@gmail.com>
Cc: <vdat2025@iitrpr.ac.in>

Dear Suvarnaganti Leela Nikhil,

Thank you for registering for **VDAT 2025**.

PFA invoice of your registration payment. Apologies for the delay. Please check the invoice and reply to this email if you find any discrepancy with details.

The full schedule of the VDAT'25, both Tutorials and Technical program are available on the conference webpage. We are excited to share that during VDAT'25 many distinguished speakers both from industry and academia are going to deliver keynotes/invited-talks/panel-discussions.
Keep checking the conference page for latest updates.

Hope you have planned your travel. We look forward to meeting you during the conference.

Regards,
VDAT'25 Registration Chair